

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.eventRelease`

1.1.0-SNAPSHOT

September 2026



OSGi®

Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
113	Event Admin Service Specification	7
113.1	Introduction	7
113.2	Event Admin Architecture	8
113.3	The Event	9
113.4	Event Handler	10
113.5	Event Publisher	12
113.6	Specific Events	13
113.7	Event Admin Service	15
113.8	Reliability	17
113.9	Interoperability with Native Applications	17
113.10	Capabilities	17
113.11	Security	18
113.12	org.osgi.service.event	19
113.13	org.osgi.service.event.annotations	28
113.14	org.osgi.service.event.propertytypes	28

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

113 Event Admin Service Specification

Version 1.4

113.1 Introduction

Nearly all the bundles in an OSGi framework must deal with events, either as an event publisher or as an event handler. So far, the preferred mechanism to disperse those events have been the service interface mechanism.

Dispatching events for a design related to X, usually involves a service of type XListener. However, this model does not scale well for fine grained events that must be dispatched to many different handlers. Additionally, the dynamic nature of the OSGi environment introduces several complexities because both event publishers and event handlers can appear and disappear at any time.

The Event Admin service provides an inter-bundle communication mechanism. It is based on a event *publish* and *subscribe* model, popular in many message based systems.

This specification defines the details for the participants in this event model.

113.1.1

Essentials

- *Simplifications* - The model must significantly simplify the process of programming an event source and an event handler.
- *Dependencies* - Handle the myriad of dependencies between event sources and event handlers for proper cleanup.
- *Synchronicity* - It must be possible to deliver events asynchronously or synchronously with the caller.
- *Event Window* - Only event handlers that are active when an event is published must receive this event, handlers that register later must not see the event.
- *Performance* - The event mechanism must impose minimal overhead in delivering events.
- *Selectivity* - Event listeners must only receive notifications for the event types for which they are interested
- *Reliability* - The Event Admin must ensure that events continue to be delivered regardless the quality of the event handlers.
- *Security* - Publishing and receiving events are sensitive operations that must be protected per event type.
- *Extensibility* - It must be possible to define new event types with their own data types.
- *Native Code* - Events must be able to be passed to native code or come from native code.
- *OSGi Events* - The OSGi Framework, as well as a number of OSGi services, already have number of its own events defined. For uniformity of processing, these have to be mapped into generic event types.

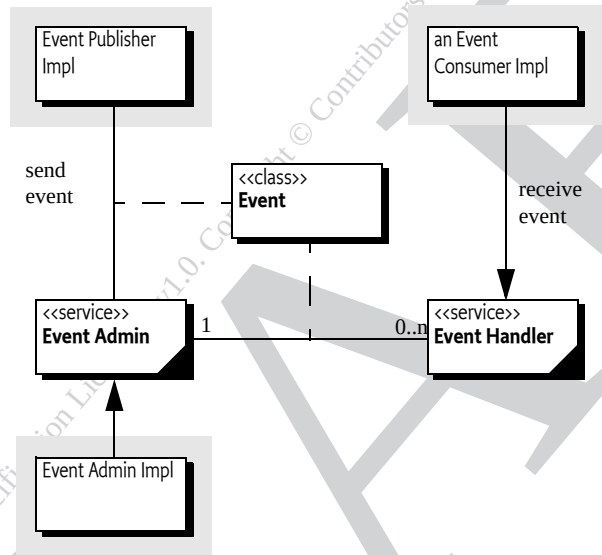
113.1.2

Entities

- *Event* - An Event object has a topic and a Dictionary object that contains the event properties. It is an immutable object.
- *Event Admin* - The service that provides the publish and subscribe model to Event Handlers and Event Publishers.

- *Event Handler* - A service that receives and handles Event objects.
- *Event Publisher* - A bundle that sends event through the Event Admin service.
- *Event Subscriber* - Another name for an Event Handler.
- *Topic* - The name of an Event type.
- *Event Properties* - The set of properties that is associated with an Event.

Figure 113.1

The Event Admin service *org.osgi.service.event* package

113.1.3

Synopsis

The Event Admin service provides a place for bundles to publish events, regardless of their destination. It is also used by Event Handlers to subscribe to specific types of events.

Events are published under a topic, together with a number of event properties. Event Handlers can specify a filter to control the Events they receive on a very fine grained basis.

113.1.4

What To Read

- *Architects* - The *Event Admin Architecture* on page 8 provides an overview of the Event Admin service.
- *Event Publishers* - The *Event Publisher* on page 12 provides an introduction of how to write an Event Publisher. The *Event Admin Architecture* on page 8 provides a good overview of the design.
- *Event Subscribers/Handlers* - The *Event Handler* on page 10 provides the rules on how to subscribe and handle events.

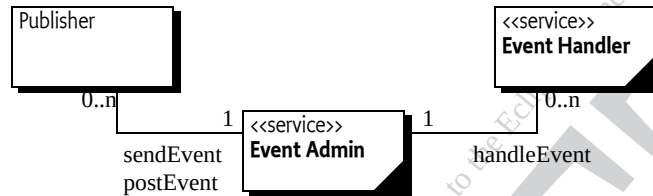
113.2

Event Admin Architecture

The Event Admin is based on the *Publish-Subscribe* pattern. This pattern decouples sources from their handlers by interposing an *event channel* between them. The publisher posts events to the channel, which identifies which handlers need to be notified and then takes care of the notification process. This model is depicted in Figure 113.2.

Figure 113.2

Channel Pattern



In this model, the event source and event handler are completely decoupled because neither has any direct knowledge of the other. The complicated logic of monitoring changes in the event publishers and event handlers is completely contained within the event channel. This is highly advantageous in an OSGi environment because it simplifies the process of both sending and receiving events.

113.3 The Event

Events have the following attributes:

- *Topic* - A topic that defines what happened. For example, when a bundle is started an event is published that has a topic of `org.osgi/framework/BundleEvent/STARTED`.
- *Properties* - Zero or more properties that contain additional information about the event. For example, the previous example event has a property of `bundle.id` which is set to a Long object, among other properties.

113.3.1 Topics

The topic of an event defines the *type* of the event. It is fairly granular in order to give handlers the opportunity to register for just the events they are interested in. When a topic is designed, its name should not include any other information, such as the publisher of the event or the data associated with the event, those parts are intended to be stored in the event properties.

The topic is intended to serve as a first-level filter for determining which handlers should receive the event. Event Admin service implementations use the structure of the topic to optimize the dispatching of the events to the handlers.

Topics are arranged in a hierarchical namespace. Each level is defined by a token and levels are separated by solidi ('/' \u002F). More precisely, the topic must conform to the following grammar:

```
topic ::= token ( '/' token ) *    // See General Syntax Definitions in Core
```

Topics should be designed to become more specific when going from left to right. Handlers can provide a prefix that matches a topic, using the preferred order allows a handler to minimize the number of prefixes it needs to register.

Topics are case-sensitive. As a convention, topics should follow the reverse domain name scheme used by Java packages to guarantee uniqueness. The separator must be a solidus ('/' \u002F) instead of the full stop ('.' \u002E).

This specification uses the convention `fully/qualified/package/ClassName/ACTION`. If necessary, a pseudo-class-name is used.

113.3.2 Properties

Information about the actual event is provided as properties. The property name is a case-sensitive string and the value can be any object. Although any Java object can be used as a property value, only String objects and the eight primitive types (plus their wrappers) should be used. Other types cannot be passed to handlers that reside external from the Java VM.

Another reason that arbitrary classes should not be used is the mutability of objects. If the values are not immutable, then any handler that receives the event could change the value. Any handlers that received the event subsequently would see the altered value and not the value as it was when the event was sent.

The topic of the event is available as a property with the key `EVENT_TOPIC`. This allows filters to include the topic as a condition if necessary.

113.3.3

High Performance

An event processing system can become a bottleneck in large systems. One expensive aspect of the Event object is its properties and its immutability. This combination requires the Event object to create a copy of the properties for each object. There are many situations where the same properties are dispatched through Event Admin, the topic is then used to signal the information. Creating the copy of the properties can therefore take unnecessary CPU time and memory. However, the immutability of the Event object requires the properties to be immutable.

For this reason, this specification also provides an immutable Map with the Event Properties class. This class implements an immutable map that is recognized and trusted by the Event object to not mutate. Using an Event Properties object allows a client to create many different Event objects with different topics but sharing the same properties object.

The following example shows how an event poster can limit the copying of the properties.

```
void foo(EventAdmin eventAdmin) {
    Map<String, Object> props = new HashMap<String, Object>();
    props.put("foo", 1);
    EventProperties eventProps = new EventProperties( props);

    for ( int i=0; i<1000; i++)
        eventAdmin.postEvent( new Event( "my/topic/" + i, eventProps));
}
```

113.4

Event Handler

Event handlers must be registered as services with the OSGi framework under the object class `org.osgi.service.event.EventHandler`.

Event handlers should be registered with a property (constant from the EventConstants class) `EVENT_TOPIC`. The value being a String, String[] or Collection<String> object that describes which topics the handler is interested in. A wildcard asterisk ('*' \u002A) may be used as the last token of a topic name, for example `com/action/*`. This matches any topic that shares the same first tokens. For example, `com/action/*` matches `com/action/listen`.

Event Handlers which have not specified the `EVENT_TOPIC` service property must not receive events.

The value of each entry in the `EVENT_TOPIC` service registration property must conform to the following grammar:

```
topic-scope ::= '*' | ( topic '/*'? )
```

The `EventTopics` component property type can be used for this property on Declarative Services components.

Event handlers can also be registered with a service property named `EVENT_FILTER`. The value of this property must be a string containing a Framework filter specification. Any of the event's properties can be used in the filter expression.

```
event-filter ::= filter // See Filter Syntax in Core
```

Each Event Handler is notified for any event which belongs to the topics the handler has expressed an interest in. If the handler has defined a [EVENT_FILTER](#) service property then the event properties must also match the filter expression. If the filter is an error, then the Event Admin service should log a warning and further ignore the Event Handler. The [EventFilter](#) component property type can be used for this property on Declarative Services components.

For example, a bundle wants to see all Log Service events with a level of WARNING or ERROR, but it must ignore the INFO and DEBUG events. Additionally, the only events of interest are when the bundle symbolic name starts with com.acme.

```
public AcmeWatchDog implements BundleActivator,
    EventHandler {
    final static String [] topics = new String[] {
        "org.osgi/service/log/LogEntry/LOG_WARNING",
        "org.osgi/service/log/LogEntry/LOG_ERROR" };

    public void start(BundleContext context) {
        Dictionary d = new Hashtable();
        d.put(EventConstants.EVENT_TOPIC, topics );
        d.put(EventConstants.EVENT_FILTER,
            "(bundle.symbolicName=com.acme.*)" );
        context.registerService( EventHandler.class.getName(),
            this, d );
    }
    public void stop( BundleContext context) {}

    public void handleEvent(Event event ) {
        //...
    }
}
```

If there are multiple Event Admin services registered with the Framework then all Event Admin services must send their published events to all registered Event Handlers.

113.4.1 Ordering

In the default case, an Event Handler will receive posted (asynchronous) events from a single thread in the same order as they were posted. Maintaining this ordering guarantee requires the Event Admin to serialize the delivery of events instead of, for example, delivering the events on different worker threads. There are many scenarios where this ordering is not really required. For this reason, an Event Handler can signal to the Event Admin that events can be delivered out of order. This is notified with the [EVENT_DELIVERY](#) service property. This service property can be used in the following way:

- Not set or set to both - The Event Admin must deliver the events in the proper order.
- [DELIVERY_ASYNC_ORDERED](#) - Events must be delivered in order.
- [DELIVERY_ASYNC_UNORDERED](#) - Allow the events to be delivered in any order.

The [EventDelivery](#) component property type can be used for this property on Declarative Services components.

113.5 Event Publisher

To fire an event, the event source must retrieve the Event Admin service from the OSGi service registry. Then it creates the event object and calls one of the Event Admin service's methods to fire the event either synchronously or asynchronously.

The following example is a class that publishes a time event every 60 seconds.

```
public class TimerEvent extends Thread
    implements BundleActivator {
    Hashtable      time = new Hashtable();
    ServiceTracker tracker;

    public TimerEvent() { super("TimerEvent"); }

    public void start(BundleContext context ) {
        tracker = new ServiceTracker(context,
            EventAdmin.class.getName(), null );
        tracker.open();
        start();
    }

    public void stop( BundleContext context ) {
        interrupt();
        tracker.close();
    }

    public void run() {
        while ( ! Thread.interrupted() ) try {
            Calendar c = Calendar.getInstance();
            set(c,Calendar.MINUTE,"minutes");
            set(c,Calendar.HOUR,"hours");
            set(c,Calendar.DAY_OF_MONTH,"day");
            set(c,Calendar.MONTH,"month");
            set(c,Calendar.YEAR,"year");

            EventAdmin ea =
                (EventAdmin) tracker.getService();
            if ( ea != null )
                ea.sendEvent(new Event("com/acme/timer",
                    time ));
            Thread.sleep(60000-c.get(Calendar.SECOND)*1000);
        } catch( InterruptedException e ) {
            return;
        }
    }

    void set( Calendar c, int field, String key ) {
        time.put( key, new Integer(c.get(field)) );
    }
}
```

113.6Specific Events

113.6.1General Conventions

Some handlers are more interested in the contents of an event rather than what actually happened. For example, a handler wants to be notified whenever an Exception is thrown anywhere in the system. Both Framework Events and Log Entry events may contain an exception that would be of interest to this hypothetical handler. If both Framework Events and Log Entries use the same property names then the handler can access the Exception in exactly the same way. If some future event type follows the same conventions then the handler can receive and process the new event type even though it had no knowledge of it when it was compiled.

The following properties are suggested as conventions. When new event types are defined they should use these names with the corresponding types and values where appropriate. These values should be set only if they are not null

A list of these property names can be found in the following table.

Table 113.1General property names for events

Name	Type	Notes
BUNDLE_SIGNER	String Collection<String>	A bundle's signers DN
BUNDLE_VERSION	Version	A bundle's version
BUNDLE_SYMBOLICNAME	String	A bundle's symbolic name
EVENT	Object	The actual event object. Used when rebroadcasting an event that was sent via some other event mechanism
EXCEPTION	Throwable	An exception or error
EXCEPTION_MESSAGE	String	Must be equal to exception.getMessage().
EXCEPTION_CLASS	String	Must be equal to the name of the Exception class.
MESSAGE	String	A human-readable message that is usually not localized.
SERVICE	Service Reference	A Service Reference
SERVICE_ID	Long	A service's id
SERVICE_OBJECTCLASS	String[]	A service's objectClass
SERVICE_PID	String Collection<String>	A service's persistent identity. A PID that is specified with a String[] must be coerced into a Collection<String>.
TIMESTAMP	Long	The time when the event occurred, as reported by System.currentTimeMillis()

The topic of an OSGi event is constructed by taking the fully qualified name of the event class, substituting a solidus ('/' \u002F)for every full stop, and appending a solidus followed by the name of the constant that defines the event type. For example, the topic of

BundleEvent.STARTED

Event becomes

org/osgi/framework/BundleEvent/STARTED

If a type code for the event is unknown then the event must be ignored.

113.6.2 OSGi Events

In order to present a consistent view of all the events occurring in the system, the existing Framework-level events are mapped to the Event Admin's publish-subscribe model. This allows event subscribers to treat framework events exactly the same as other events.

It is the responsibility of the Event Admin service implementation to map these Framework events to its queue.

The properties associated with the event depends on its class as outlined in the following sections.

113.6.3 Framework Event

Framework Events must be delivered asynchronously with a topic of:

```
org/osgi/framework/FrameworkEvent/<event type>
```

The following event types are supported:

STARTED
ERROR
PACKAGES_REFRESHED
STARTLEVEL_CHANGED
WARNING
INFO

Other events are ignored, no event will be send by the Event Admin. The following event properties must be set for a Framework Event.

- `event` - (FrameworkEvent) The original event object.

If the FrameworkEvent `getBundle` method returns a non-null value, the following fields must be set:

- `bundle.id` - (Long) The source's bundle id.
- `bundle.symbolicName` - (String) The source bundle's symbolic name. Only set if the bundle's symbolic name is not null.
- `bundle.version` - (Version) The version of the bundle, if set.
- `bundle.signer` - (String|Collection<String>) The DNs of the signers.
- `bundle` - (Bundle) The source bundle.

If the FrameworkEvent `getThrowable` method returns a non-null value:

- `exception.class` - (String) The fully-qualified class name of the attached Exception.
- `exception.message` - (String) The message of the attached exception. Only set if the Exception message is not null.
- `exception` - (Throwable) The Exception returned by the `getThrowable` method.

113.6.4 Bundle Event

Framework Events must be delivered asynchronously with a topic of:

```
org/osgi/framework/BundleEvent/<event type>
```

The following event types are supported:

INSTALLED
STARTED
STOPPED

UPDATED
UNINSTALLED
RESOLVED
UNRESOLVED

Unknown events must be ignored.

The following event properties must be set for a Bundle Event. If listeners require synchronous delivery then they should register a Synchronous Bundle Listener with the Framework.

- event - (BundleEvent) The original event object.
- bundle.id - (Long) The source's bundle id.
- bundle.symbolicName - (String) The source bundle's symbolic name. Only set if the bundle's symbolic name is not null.
- bundle.version - (Version) The version of the bundle, if set.
- bundle.signer - (String|Collection<String>) The DNs of the signers.
- bundle - (Bundle) The source bundle.

113.6.5

Service Event

Service Events must be delivered asynchronously with the topic:

`org.osgi/framework/ServiceEvent/<event type>`

The following event types are supported:

REGISTERED
MODIFIED
UNREGISTERING

Unknown events must be ignored.

- event - (ServiceEvent) The original Service Event object.
- service - (ServiceReference) The result of the `getServiceReference` method
- service.id - (Long) The service's ID.
- service.pid - (String or Collection<String>) The service's persistent identity. Only set if not null. If the PID is specified as a `String[]` then it must be coerced into a `Collection<String>`.
- service.objectClass - (String[]) The service's object class.

113.6.6

Other Event Sources

Several OSGi service specifications define their own event model. It is the responsibility of these services to map their events to Event Admin events. Event Admin is seen as a core service that will be present in most devices. However, if there is no Event Admin service present, applications are not mandated to buffer events.

113.7

Event Admin Service

The Event Admin service must be registered as a service with the object class `org.osgi.service.event.EventAdmin`. Multiple Event Admin services can be registered. Publishers should publish their event on the Event Admin service with the highest value for the `SERVICE_RANKING` service property. This is the service selected by the `getServiceReference` method.

The Event Admin service is responsible for tracking the registered handlers, handling event notifications and providing at least one thread for asynchronous event delivery.

113.7.1 Synchronous Event Delivery

Synchronous event delivery is initiated by the `sendEvent` method. When this method is invoked, the Event Admin service determines which handlers must be notified of the event and then notifies each one in turn. The handlers can be notified in the caller's thread or in an event-delivery thread, depending on the implementation. In either case, all notifications must be completely handled before the `sendEvent` method returns to the caller.

Synchronous event delivery is significantly more expensive than asynchronous delivery. All things considered equal, the asynchronous delivery should be preferred over the synchronous delivery.

Callers of this method will need to be coded defensively and assume that synchronous event notifications could be handled in a separate thread. That entails that they must not be holding any monitors when they invoke the `sendEvent` method. Otherwise they significantly increase the likelihood of deadlocks because Java monitors are not reentrant from another thread by definition. Not holding monitors is good practice even when the event is dispatched in the same thread.

113.7.2 Asynchronous Event Delivery

Asynchronous event delivery is initiated by the `postEvent` method. When this method is invoked, the Event Admin service must determine which handlers are interested in the event. By collecting this list of handlers during the method invocation, the Event Admin service ensures that only handlers that were registered at the time the event was posted will receive the event notification. This is the same as described in *Delivering Events* of ???.

The Event Admin service can use more than one thread to deliver events. If it does then it must guarantee that each handler receives the events in the same order as the events were posted, unless this handler allows unordered deliver, see *Ordering* on page 11 . This ensures that handlers see events in their expected order. For example, for some handlers it would be an error to see a destroyed event before the corresponding created event.

Before notifying each handler, the event delivery thread must ensure that the handler is still registered in the service registry. If it has been unregistered then the handler must not be notified.

113.7.3 Order of Event Delivery

Asynchronous events are delivered in the order in which they arrive in the event queue. Thus if two events are posted by the same thread then they will be delivered in the same order (though other events may come between them). However, if two or more events are posted by different threads then the order in which they arrive in the queue (and therefore the order in which they are delivered) will depend very much on subtle timing issues. The event delivery system cannot make any guarantees in this case. An Event Handler can indicate that the ordering is not relevant, allowing the Event Admin to more aggressively parallelize the event deliver, see *Ordering* on page 11 .

Synchronous events are delivered as soon as they are sent. If two events are sent by the same thread, one after the other, then they must be guaranteed to be processed serially and in the same order. However, if two events are sent by different threads then no guarantees can be made. The events can be processed in parallel or serially, depending on whether or not the Event Admin service dispatches synchronous events in the caller's thread or in a separate thread.

Note that if the actions of a handler trigger a synchronous event, then the delivery of the first event will be paused and delivery of the second event will begin. Once delivery of the second event has completed, delivery of the first event will resume. Thus some handlers may observe the second event before they observe the first one.

113.8 Reliability

113.8.1 Exceptions in callbacks

If a handler throws an Exception during delivery of an event, it must be caught by the Event Admin service and handled in some implementation specific way. If a Log Service is available the exception should be logged. Once the exception has been caught and dealt with, the event delivery must continue with the next handlers to be notified, if any.

As the Log Service can also forward events through the Event Admin service there is a potential for a loop when an event is reported to the Log Service.

113.8.2 Dealing with Stalled Handlers

Event handlers should not spend too long in the `handleEvent` method. Doing so will prevent other handlers in the system from being notified. If a handler needs to do something that can take a while, it should do it in a different thread.

An event admin implementation can attempt to detect stalled or deadlocked handlers and deal with them appropriately. Exactly how it deals with this situation is left as implementation specific. One allowed implementation is to mark the current event delivery thread as invalid and spawn a new event delivery thread. Event delivery must resume with the next handler to be notified.

Implementations can choose to deny list any handlers that they determine are misbehaving. Deny listed handlers must not be notified of any events. If a handler is deny listed, the event admin should log a message that explains the reason for it.

113.9 Interoperability with Native Applications

Implementations of the Event Admin service can support passing events to, and/or receiving events from native applications.

If the implementation supports native interoperability, it must be able to pass the topic of the event and its properties to/from native code. Implementations must be able to support property values of the following types:

- String objects, including full Unicode support
- Integer, Long, Byte, Short, Float, Double, Boolean, Character objects
- Single-dimension arrays of the above types (including String)
- Single-dimension arrays of Java's eight primitive types (int, long, byte, short, float, double, boolean, char)

Implementations can support additional types. Property values of unsupported types must be silently discarded.

113.10 Capabilities

113.10.1 osgi.implementation Capability

The Event Admin implementation bundle must provide the `osgi.implementation` capability with the name `EVENT_ADMIN_IMPLEMENTATION`. This capability can be used by provisioning tools and during resolution to ensure that an Event Admin implementation is present. The capability must also declare a uses constraint for the `org.osgi.service.event` package and provide the version of this specification:

```
Provide-Capability: osgi.implementation;  
    osgi.implementation="osgi.event";  
    uses:="org.osgi.service.event";  
    version:Version="  
        1.4  
    "
```

The [RequireEventAdmin](#) annotation can be used to require this capability.

This capability must follow the rules defined for the ???.

113.10.2 **osgi.service Capability**

The bundle providing the Event Admin service must provide a capability in the `osgi.service` namespace representing this service. This capability must also declare a uses constraint for the `org.osgi.service.event` package:

```
Provide-Capability: osgi.service;  
    objectClass:List<String>="org.osgi.service.event.EventAdmin";  
    uses:="org.osgi.service.event"
```

This capability must follow the rules defined for the ???.

113.11 **Security**

113.11.1 **Topic Permission**

The `TopicPermission` class allows fine-grained control over which bundles may post events to a given topic and which bundles may receive those events.

The target parameter for the permission is the topic name. `TopicPermission` classes uses a wildcard matching algorithm similar to the `BasicPermission` class, except that solidi (`'/'` `\u002F`) are used as separators instead of full stop characters. For example, a name of `a/b/*` implies `a/b/c` but not `x/y/z` or `a/b`.

There are two available actions: `PUBLISH` and `SUBSCRIBE`. These control a bundle's ability to either publish or receive events, respectively. Neither one implies the other.

113.11.2 **Required Permissions**

Bundles that need to register an event handler must be granted `ServicePermission[org.osgi.service.event.EventHandler, REGISTER]`. In addition, handlers require `TopicPermission[ <topic>, SUBSCRIBE ]` for each topic they want to be notified about.

Bundles that need to publish an event must be granted `ServicePermission[ org.osgi.service.event.EventAdmin, GET]` so that they may retrieve the Event Admin service and use it. In addition, event sources require `TopicPermission[ <topic>, PUBLISH]` for each topic they want to send events to.

Bundles that need to iterate the handlers registered with the system must be granted `ServicePermission[org.osgi.service.event.EventHandler, GET]` to retrieve the event handlers from the service registry.

Only a bundle that contains an Event Admin service implementation should be granted `ServicePermission[ org.osgi.service.event.EventAdmin, REGISTER]` to register the event channel admin service.

113.11.3 Security Context During Event Callbacks

During an event notification, the Event Admin service's Protection Domain will be on the stack above the handler's Protection Domain. In the case of a synchronous event, the event publisher's protection domain can also be on the stack.

Therefore, if a handler needs to perform a secure operation using its own privileges, it must invoke the `doPrivileged` method to isolate its security context from that of its caller.

The event delivery mechanism must not wrap event notifications in a `doPrivileged` call.

113.12 org.osgi.service.event

Event Admin Package Version 1.4.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.event; version="[1.4,2.0)"}

```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.event; version="[1.4,1.5)"}

```

113.12.1 Summary

- `Event` - An event.
- `EventAdmin` - The Event Admin service.
- `EventConstants` - Defines standard names for {@code EventHandler} properties.
- `EventHandler` - Listener for Events.
- `EventProperties` - The properties for an {@code Event}.
- `TopicPermission` - A bundle's authority to publish or subscribe to event on a topic.

113.12.2 public class Event

An event. {@code Event} objects are delivered to {@code EventHandler} services which subscribe to the topic of the event.

Concurrency Immutable

113.12.2.1 public Event(String topic, Map<String, ?> properties)

topic The topic of the event.

properties The event's properties (may be {@code null}). A property whose key is not of type {@code String} will be ignored. If the specified properties is an {@code EventProperties} object, then it will be directly used. Otherwise, a copy of the specified properties is made.

- Constructs an event.

Throws `IllegalArgumentException` – If topic is not a valid topic name.

Since @since 1.2

113.12.2.2 public Event(String topic, Dictionary<String, ?> properties)

topic The topic of the event.

properties The event's properties (may be {@code null}). A property whose key is not of type {@code String} will be ignored. A copy of the specified properties is made.

- Constructs an event.

Throws `IllegalArgumentException`– If topic is not a valid topic name.

113.12.2.3 **public final boolean containsProperty(String name)**

name The name of the property.

- Indicate the presence of an event property. The event topic is present using the property name "event.topics".

Returns {@code true} if a property with the specified name is in the event. This property may have a {@code null} value. {@code false} otherwise.

Since @since 1.3

113.12.2.4 **public boolean equals(Object object)**

object The {@code Event} object to be compared.

- Compares this {@code Event} object to another object.

An event is considered to be **equal to** another event if the topic is equal and the properties are equal. The properties are compared using the {@code java.util.Map.equals()} rules which includes identity comparison for array values.

Returns {@code true} if {@code object} is a {@code Event} and is equal to this object; {@code false} otherwise.

113.12.2.5 **public final Object getProperty(String name)**

name The name of the property to retrieve.

- Retrieve the value of an event property. The event topic may be retrieved with the property name "event.topics".

Returns The value of the property, or {@code null} if not found.

113.12.2.6 **public final String[][] getPropertyNames()**

- Returns a list of this event's property names. The list will include the event topic property name "event.topics".

Returns A non-empty array with one element per property.

113.12.2.7 **public final String getTopic()**

- Returns the topic of this event.

Returns The topic of this event.

113.12.2.8 **public int hashCode()**

- Returns a hash code value for this object.

Returns An integer which is a hash code value for this object.

113.12.2.9 **public final boolean matches(Filter filter)**

filter The filter to test.

- Tests this event's properties against the given filter using a case sensitive match.

Returns true If this event's properties match the filter, false otherwise.

113.12.2.10 public String toString()

- Returns the string representation of this event.

Returns The string representation of this event.

113.12.3 public abstract EventAdmin

The Event Admin service. Bundles wishing to publish events must obtain the Event Admin service and call one of the event delivery methods.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

113.12.3.1 public abstract void postEvent(Event event)

event The event to send to all listeners which subscribe to the topic of the event.

- Initiate asynchronous, ordered delivery of an event. This method returns to the caller before delivery of the event is completed. Events are delivered in the order that they are received by this method.

Throws `SecurityException`—If the caller does not have {@code TopicPermission[topic,PUBLISH]} for the topic specified in the event.

113.12.3.2 public abstract void sendEvent(Event event)

event The event to send to all listeners which subscribe to the topic of the event.

- Initiate synchronous delivery of an event. This method does not return to the caller until delivery of the event is completed.

Throws `SecurityException`—If the caller does not have {@code TopicPermission[topic,PUBLISH]} for the topic specified in the event.

113.12.4 public abstract EventConstants

Defines standard names for {@code EventHandler} properties.

Provider Type Consumers of this API must not implement this type

113.12.4.1 public static final String BUNDLE = "bundle"

The Bundle object of the bundle relevant to the event. The type of the value for this event property is {@code Bundle}.

Since @since 1.1

113.12.4.2 public static final String BUNDLE_ID = "bundle.id"

The Bundle id of the bundle relevant to the event. The type of the value for this event property is {@code Long}.

Since @since 1.1

113.12.4.3 public static final String BUNDLE_SIGNER = "bundle.signer"

The Distinguished Names of the signers of the bundle relevant to the event. The type of the value for this event property is {@code String} or {@code Collection} of {@code String}.

113.12.4.4 public static final String BUNDLE_SYMBOLICNAME = "bundle.symbolicName"

The Bundle Symbolic Name of the bundle relevant to the event. The type of the value for this event property is {@code String}.

113.12.4.5 public static final String BUNDLE_VERSION = "bundle.version"

The version of the bundle relevant to the event. The type of the value for this event property is {@link Version}.

Since @since 1.2

113.12.4.6 public static final String DELIVERY_ASYNC_ORDERED = "async.ordered"

Event Handler delivery quality value specifying the Event Handler requires asynchronously delivered events be delivered in order. Ordered delivery is the default for asynchronously delivered events.

This delivery quality value is mutually exclusive with {@link #DELIVERY_ASYNC_UNORDERED}. However, if both this value and {@link #DELIVERY_ASYNC_UNORDERED} are specified for an event handler, this value takes precedence.

See Also EVENT_DELIVERY

Since @since 1.3

113.12.4.7 public static final String DELIVERY_ASYNC_UNORDERED = "async.unordered"

Event Handler delivery quality value specifying the Event Handler does not require asynchronously delivered events be delivered in order. This may allow an Event Admin implementation to optimize asynchronous event delivery by relaxing ordering requirements.

This delivery quality value is mutually exclusive with {@link #DELIVERY_ASYNC_ORDERED}. However, if both this value and {@link #DELIVERY_ASYNC_ORDERED} are specified for an event handler, {@link #DELIVERY_ASYNC_ORDERED} takes precedence.

See Also EVENT_DELIVERY

Since @since 1.3

113.12.4.8 public static final String EVENT = "event"

The forwarded event object. Used when rebroadcasting an event that was sent via some other event mechanism. The type of the value for this event property is {@code Object}.

113.12.4.9 public static final String EVENT_ADMIN_IMPLEMENTATION = "osgi.event"

The name of the implementation capability for the Event Admin specification

Since @since 1.4

113.12.4.10 public static final String EVENT_ADMIN_SPECIFICATION_VERSION = "1.4"

The version of the implementation capability for the Event Admin specification

Since @since 1.4

113.12.4.11 public static final String EVENT_DELIVERY = "event.delivery"

Service Registration property specifying the delivery qualities requested by an Event Handler service.

Event handlers MAY be registered with this property. Each value of this property is a string specifying a delivery quality for the Event handler.

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection<String>}.

See Also DELIVERY_ASYNC_ORDERED, DELIVERY_ASYNC_UNORDERED

Since @since 1.3

113.12.4.12 public static final String EVENT_FILTER = "event.filter"

Service Registration property specifying a filter to further select {@code Event}s of interest to an Event Handler service.

Event handlers MAY be registered with this property. The value of this property is a string containing an LDAP-style filter specification. Any of the event's properties may be used in the filter expression. Each event handler is notified for any event which belongs to the topics in which the handler has expressed an interest. If the event handler is also registered with this service property, then the properties of the event must also match the filter for the event to be delivered to the event handler.

If the filter syntax is invalid, then the Event Handler must be ignored and a warning should be logged.

The value of this property must be of type {@code String}.

See Also Event , Filter

113.12.4.13 public static final String EVENT_TOPIC = "event.topics"

Service registration property specifying the {@code Event} topics of interest to an Event Handler service.

Event handlers SHOULD be registered with this property. Each value of this property is a string that describe the topics in which the handler is interested. An asterisk (*) may be used as a trailing wildcard. Event Handlers which do not have a value for this property must not receive events. More precisely, the value of each string must conform to the following grammar:

```
topic-description := '*' | topic ( '/' '*' ) ?  
topic := token ( '/' token ) *
```

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection<String>}.

See Also Event

113.12.4.14 public static final String EXCEPTION = "exception"

An exception or error. The type of the value for this event property is {@code Throwable}.

113.12.4.15 public static final String EXCEPTION_CLASS = "exception.class"

The name of the exception type. Must be equal to the name of the class of the exception in the event property {@link #EXCEPTION}. The type of the value for this event property is {@code String}.

Since @since 1.1

113.12.4.16 public static final String EXCEPTION_MESSAGE = "exception.message"

The exception message. Must be equal to the result of calling {@code getMessage()} on the exception in the event property {@link #EXCEPTION}. The type of the value for this event property is {@code String}.

113.12.4.17 public static final String EXCEPTION_CLASS = "exception.class"

This constant was released with an incorrectly spelled name. It has been replaced by {@link #EXCEPTION_CLASS}

Deprecated @deprecated As of 1.1. Replaced by {@link #EXCEPTION_CLASS}.

113.12.4.18 public static final String MESSAGE = "message"

A human-readable message that is usually not localized. The type of the value for this event property is {@code String}.

113.12.4.19 **public static final String SERVICE = "service"**

A service reference. The type of the value for this event property is {@link ServiceReference}.

113.12.4.20 **public static final String SERVICE_ID = "service.id"**

A service's id. The type of the value for this event property is {@code Long}.

113.12.4.21 **public static final String SERVICE_OBJECTCLASS = "service.objectClass"**

A service's objectClass. The type of the value for this event property is {@code String[]}.

113.12.4.22 **public static final String SERVICE_PID = "service.pid"**

A service's persistent identity. The type of the value for this event property is {@code String} or {@code Collection} of {@code String}.

113.12.4.23 **public static final String TIMESTAMP = "timestamp"**

The time when the event occurred, as reported by {@code System.currentTimeMillis()}. The type of the value for this event property is {@code Long}.

113.12.5 **public abstract EventHandler**

Listener for Events.

{@code EventHandler} objects are registered with the Framework service registry and are notified with an {@code Event} object when an event is sent or posted.

{@code EventHandler} objects can inspect the received {@code Event} object to determine its topic and properties.

{@code EventHandler} objects must be registered with a service property {@link EventConstants#EVENT_TOPIC} whose value is the list of topics in which the event handler is interested.

For example:

```
String[] topics = new String[] { "com/ismv/*" };
Hashtable ht = new Hashtable();
ht.put(EventConstants.EVENT_TOPIC, topics);
context.registerService(EventHandler.class.getName(), this, ht);
```

Event Handler services can also be registered with an {@link EventConstants#EVENT_FILTER} service property to further filter the events. If the syntax of this filter is invalid, then the Event Handler must be ignored by the Event Admin service. The Event Admin service should log a warning.

Security Considerations. Bundles wishing to monitor {@code Event} objects will require {@code ServicePermission[EventHandler,REGISTER]} to register an {@code EventHandler} service. The bundle must also have {@code TopicPermission[topic,SUBSCRIBE]} for the topic specified in the event in order to receive the event.

See Also Event

Concurrency Thread-safe

113.12.5.1 **public abstract void handleEvent(Event event)**

event The event that occurred.

- Called by the {@link EventAdmin} service to notify the listener of an event.

113.12.6 **public class EventProperties
implements Map<String, Object>**

The properties for an {@link Event}. An event source can create an EventProperties object if it needs to reuse the same event properties for multiple events.

The keys are all of type {@code String}. The values are of type {@code Object}. The key "event.topics" is ignored as event topics can only be set when an {@link Event} is constructed.

Once constructed, an EventProperties object is unmodifiable. However, the values of the map used to construct an EventProperties object are still subject to modification as they are not deeply copied.

Since @since 1.3

Concurrency Immutable

113.12.6.1 **public EventProperties(Map<String, ?> properties)**

properties The properties to use for this EventProperties object (may be {@code null}).

- Create an EventProperties from the specified properties.

The specified properties will be copied into this EventProperties. Properties whose key is not of type {@code String} will be ignored. A property with the key "event.topics" will be ignored.

113.12.6.2 **public void clear()**

- This method throws {@link UnsupportedOperationException}.

Throws UnsupportedOperationException—if called.

113.12.6.3 **public boolean containsKey(Object name)**

name The property name.

- Indicates if the specified property is present.

Returns {@code true} If the property is present, {@code false} otherwise.

113.12.6.4 **public boolean containsValue(Object value)**

value The property value.

- Indicates if the specified value is present.

Returns {@code true} If the value is present, {@code false} otherwise.

113.12.6.5 **public Set<Entry<String, Object>> entrySet()**

- Return the property entries.

Returns A set containing the property name/value pairs.

113.12.6.6 **public boolean equals(Object object)**

object The {@code EventProperties} object to be compared.

- Compares this {@code EventProperties} object to another object.

The properties are compared using the {@code java.util.Map.equals()} rules which includes identity comparison for array values.

Returns {@code true} if {@code object} is a {@code EventProperties} and is equal to this object; {@code false} otherwise.

113.12.6.7 **public Object get(Object name)**

name The name of the specified property.

- Return the value of the specified property.

Returns The value of the specified property.

113.12.6.8 **public int hashCode()**

- Returns a hash code value for this object.

Returns An integer which is a hash code value for this object.

113.12.6.9 **public boolean isEmpty()**

- Indicate if this properties is empty.

Returns {@code true} If this properties is empty, {@code false} otherwise.

113.12.6.10 **public Set<String> keySet()**

- Return the names of the properties.

Returns The names of the properties.

113.12.6.11 **public Object put(String key, Object value)**

- This method throws {@link UnsupportedOperationException}.

Throws UnsupportedOperationException – if called.

113.12.6.12 **public void putAll(Map<? extends String, ? extends Object> map)**

- This method throws {@link UnsupportedOperationException}.

Throws UnsupportedOperationException – if called.

113.12.6.13 **public Object remove(Object key)**

- This method throws {@link UnsupportedOperationException}.

Throws UnsupportedOperationException – if called.

113.12.6.14 **public int size()**

- Return the number of properties.

Returns The number of properties.

113.12.6.15 **public String toString()**

- Returns the string representation of this object.

Returns The string representation of this object.

113.12.6.16 **public Collection<Object> values()**

- Return the properties values.

Returns The values of the properties.

113.12.7 **public final class TopicPermission extends Permission**

A bundle's authority to publish or subscribe to event on a topic.

A topic is a slash-separated string that defines a topic.

For example:

```
org.osgi/service/foo/FooEvent/ACTION
```

{@code TopicPermission} has two actions: {@code publish} and {@code subscribe}.

Concurrency Thread-safe

113.12.7.1 **public static final String PUBLISH = "publish"**

The action string {@code publish}.

113.12.7.2 public static final String SUBSCRIBE = "subscribe"

The action string {@code subscribe}.

113.12.7.3 public TopicPermission(String name, String actions)

name Topic name.

actions {@code publish},{@code subscribe} (canonical order).

- Defines the authority to publish and/or subscribe to a topic within the EventAdmin service.

The name is specified as a slash-separated string. Wildcards may be used. For example:

```
org/osgi/service/fooFooEvent/ACTION
com/isv/*
*
```

A bundle that needs to publish events on a topic must have the appropriate {@code TopicPermission} for that topic; similarly, a bundle that needs to subscribe to events on a topic must have the appropriate {@code TopicPermission} for that topic.

113.12.7.4 public boolean equals(Object obj)

obj The object to test for equality with this {@code TopicPermission} object.

- Determines the equality of two {@code TopicPermission} objects. This method checks that specified {@code TopicPermission} has the same topic name and actions as this {@code TopicPermission} object.

Returns {@code true} if {@code obj} is a {@code TopicPermission}, and has the same topic name and actions as this {@code TopicPermission} object; {@code false} otherwise.

113.12.7.5 public String getActions()

- Returns the canonical string representation of the {@code TopicPermission} actions.

Always returns present {@code TopicPermission} actions in the following order: {@code publish},{@code subscribe}.

Returns Canonical string representation of the {@code TopicPermission} actions.

113.12.7.6 public int hashCode()

- Returns the hash code value for this object.

Returns A hash code value for this object.

113.12.7.7 public boolean implies(Permission p)

p The target permission to interrogate.

- Determines if the specified permission is implied by this object.

This method checks that the topic name of the target is implied by the topic name of this object. The list of {@code TopicPermission} actions must either match or allow for the list of the target object to imply the target {@code TopicPermission} action.

```
x/y/*,"publish" -> x/y/z,"publish" is true
*,"subscribe" -> x/y,"subscribe"  is true
*,"publish"    -> x/y,"subscribe"  is false
x/y,"publish"  -> x/y/z,"publish"  is false
```

Returns {@code true} if the specified {@code TopicPermission} action is implied by this object; {@code false} otherwise.

113.12.7.8 public PermissionCollection newPermissionCollection()

- Returns a new {@code PermissionCollection} object suitable for storing {@code TopicPermission} objects.

Returns A new {@code PermissionCollection} object.

113.13 org.osgi.service.event.annotations

Event Admin Annotations Package Version 1.4.

This package contains annotations that can be used to require the Event Admin implementation.

Bundles should not normally need to import this package as the annotations are only used at build-time.

113.13.1 Summary

- `RequireEventAdmin` - This annotation can be used to require the Event Admin implementation.

113.13.2 @RequireEventAdmin

This annotation can be used to require the Event Admin implementation. It can be used directly, or as a meta-annotation.

This annotation is applied to several of the Event Admin component property type annotations meaning that it does not normally need to be applied to Declarative Services components which use the Event Admin.

Since @since 1.4

Retention CLASS

Target TYPE, PACKAGE

113.14 org.osgi.service.event.propertytypes

Event Admin Component Property Types Package Version 1.4.

When used as annotations, component property types are processed by tools to generate Component Descriptions which are used at runtime.

Bundles wishing to use this package at runtime must list the package in the Import-Package header of the bundle's manifest.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.event.propertytypes; version="[1.4,2.0)"}
```

113.14.1 Summary

- `EventDelivery` - Component Property Type for the {@link EventConstants#EVENT_DELIVERY} service property of an {@link EventHandler} service.
- `EventFilter` - Component Property Type for the {@link EventConstants#EVENT_FILTER} service property of an {@link EventHandler} service.
- `EventTopics` - Component Property Type for the {@link EventConstants#EVENT_TOPIC} service property of an {@link EventHandler} service.

113.14.2 @EventDelivery

Component Property Type for the {@link EventConstants#EVENT_DELIVERY} service property of an {@link EventHandler} service.

This annotation can be used on an {@link EventHandler} component to declare the value of the {@link EventConstants#EVENT_DELIVERY} service property.

See Also Component Property Types

Since @since 1.4

Retention CLASS

Target TYPE

113.14.2.1 String[][] value

- Service property specifying the {@code Event} delivery qualities requested by an {@link EventHandler} service.

The supported delivery qualities are:

- {@link EventConstants#DELIVERY_ASYNC_ORDERED}
- {@link EventConstants#DELIVERY_ASYNC_UNORDERED}

Returns The requested event delivery qualities.

See Also EventConstants.EVENT_DELIVERY

113.14.3 @EventFilter

Component Property Type for the {@link EventConstants#EVENT_FILTER} service property of an {@link EventHandler} service.

This annotation can be used on an {@link EventHandler} component to declare the value of the {@link EventConstants#EVENT_FILTER} service property.

See Also Component Property Types

Since @since 1.4

Retention CLASS

Target TYPE

113.14.3.1 String value

- Service property specifying the {@code Event} filter to an {@link EventHandler} service.

Returns The event filter.

See Also EventConstants.EVENT_FILTER

113.14.4 @EventTopics

Component Property Type for the {@link EventConstants#EVENT_TOPIC} service property of an {@link EventHandler} service.

This annotation can be used on an {@link EventHandler} component to declare the values of the {@link EventConstants#EVENT_TOPIC} service property.

See Also Component Property Types

Since @since 1.4

Retention CLASS

Target TYPE

113.14.4.1**String[][] value**

- Service property specifying the {@code Event} topics of interest to an {@link EventHandler} service.

Returns The event topics.

See Also EventConstants.EVENT_TOPIC

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT